Laporan Optimasi & Penggabungan Task Workflow Conductor

Netflix Conductor OSS on Quarkus (v3.30.2-quarkus) | Tanggal: 22 Oktober 2026

1. Ringkasan Eksekutif (Executive Summary)

Dalam arsitektur Netflix Conductor, setiap task di dalam suatu workflow memerlukan operasi penyimpanan state ke Redis (TASK.*, WORKFLOW_TO_TASKS.*) serta pengindeksan dokumen ke Elasticsearch (conductor_task). Sebelumnya, terdapat 112 workflow perhitungan tarif/tagihan yang masing-masing terdiri dari 20 hingga 97 task INLINE terpisah, di mana setiap task hanya menjalankan satu baris operasi matematika sederhana.

Optimasi dilakukan dengan menggabungkan seluruh rumus berurutan di dalam masing-masing workflow perhitungan menjadi 1 task INLINE terpadu (hitung_semua_ref) tanpa mengubah logika bisnis, urutan perhitungan, maupun struktur outputParameters.

Metrik Optimasi	Sebelum Optimasi	Sesudah Optimasi	Efisiensi
Total Task di 127 Workflow	4.706 Task	163 Task	-96,5% Task
Jumlah Task di 112 WF Hitung	4.655 Task	112 Task (1/WF)	-97,6% Task
Eksekusi Task workflow_normal	~177 Task / trx	~3 Task Hitung / trx	50x Lebih Ringan
Kesesuaian Output (Equivalence)	Baseline	100% Identik	0 Mismatch

2. Analisis Penyebab Lambatnya Antrian Task Sebelum Optimasi

* Overhead I/O Redis & Elasticsearch Berlipat Ganda: Satu eksekusi workflow_normal yang memanggil subworkflow perhitungan tarif (43 task), subworkflow diskon (37 task), dan workflow_general_non_energi v4 (97 task) menghasilkan ~177 penulisan state task ke Redis dan ~177 dokumen index ke Elasticsearch hanya untuk 1 pelanggan.

* Mengapa FORK_JOIN Tidak Mempercepat Task INLINE: Task INLINE sudah berjalan sinkron di memori (< 1 ms), sedangkan task JOIN pada FORK_JOIN adalah Asynchronous System Task yang wajib masuk ke antrian Redis (conductor_queues.QUEUE.JOIN), menunggu polling SystemTaskWorker, serta memicu perebutan workflow execution lock antar cabang fork.

* Bottleneck GraalJS Context Pool: Seluruh ekspresi INLINE dievaluasi menggunakan GraalJS yang dibatasi oleh CONDUCTOR_SCRIPT_CONTEXT_POOL_SIZE.

3. Rincian Penggabungan Task per Kelompok Workflow

Kelompok / Nama Workflow	Jml WF	Task Sebelum	Task Sesudah
workflow_general_non_energi (v4)	1	97 INLINE	1 INLINE
workflow_general_non_energi (v1)	1	64 INLINE	1 INLINE
workflow_standar_billing_pecahan_plts (v1)	1	63 INLINE	1 INLINE
workflow_standar_billing_plts_rm (v1)	1	60 INLINE	1 INLINE
workflow_standar_billing_pecahan_rm_non_rm_*	32	47 INLINE	1 INLINE
workflow_standar_rm_non_rm_*	30	43 INLINE	1 INLINE
workflow_standar_perhitungan_diskon & kompor_induksi	2	37 INLINE	1 INLINE
workflow_standar_billing_pecahan_beban_blok_*	12	35 INLINE	1 INLINE
workflow_standar_beban_blok_*	12	31 INLINE	1 INLINE
Workflow Perhitungan Lainnya (Abonemen, Traksi, UJL, Koreksi)	20	20 - 43 INLINE	1 INLINE
Workflow Orchestrator (workflow_normal, mutasi_multiple_pecahan, dll.)	15	3 - 8 Task	Merge INLINE berurutan

Laporan Optimasi & Penggabungan Task Workflow Conductor

Netflix Conductor OSS on Quarkus (v3.30.2-quarkus) | Tanggal: 22 Oktober 2026

4. Mekanisme Teknis Penggabungan Rumus (100% Backward-Compatible)

* 1. Resolusi Input Workflow (${workflow.input.data.*}): Seluruh path input unik tetap didaftarkan pada inputParameters task hitung_semua_ref (sebagai w_0, w_1, dst.) sehingga tetap dievaluasi oleh ParametersUtils/JsonPath bawaan Conductor.

* 2. Isolasi Scope Tiap Rumus (IIFE): Setiap rumus dibungkus dalam fungsi terisolasi (function($) { ... })({ ... }) dan hasilnya disimpan ke dalam objek akumulator _r['<taskReferenceName>'] sesuai urutan ketergantungan asli.

* 3. Pemetaan outputParameters Workflow: Seluruh referensi ${<taskRef>.output.result} pada outputParameters diarahkan ke ${hitung_semua_ref.output.result.<taskRef>}, sehingga output JSON workflow tetap 100% identik.

```
function generatedFunction() {
  var _r = {};
  _r["flag_mutasi_mundur_ref_1"] = (function($) {
    var total=$.total_energies, batas0=$.jenis_batas0, batas1=$.jenis_batas1, pecah=$.pecah_ke;
    return (total===2 && batas0==="kwh" && batas1==="flat" && pecah===0) ? 1 : 0;
  })({"total_energies": $.w_0, "jenis_batas0": $.w_1, "jenis_batas1": $.w_2, "pecah_ke": $.w_3});

  _r["kwh_lwbp_blok_1_ref_1"] = (function($) {
    var flagBlok = Number($.flag_blok), batasBlok1 = Number($.batas_blok1);
    return flagBlok === 1 ? batasBlok1 : $.kwhlwbp;
  })({"batas_blok1": $.w_4, "flag_blok": _r["flag_mutasi_mundur_ref_1"], "kwhlwbp": $.w_6});
  // ... rumus berikutnya ...
  return _r;
}
generatedFunction();
```

5. Rekomendasi Tuning Konfigurasi Conductor Quarkus & Elasticsearch

```
# --- compose.yml (Environment Conductor) ---
CONDUCTOR_SCRIPT_CONTEXT_POOL_ENABLED=true
CONDUCTOR_SCRIPT_CONTEXT_POOL_SIZE=150
JAVA_OPTS=-Dpolyglot.engine.WarnInterpreterOnly=false -Xms1G -Xmx3G -XX:+UseG1GC

# --- config-redis-cluster-es8.properties ---
quarkus.thread-pool.core-threads=64
quarkus.thread-pool.max-threads=400
quarkus.thread-pool.growth-resistance=0
quarkus.vertx.event-loops-pool-size=16
quarkus.http.io-threads=16
quarkus.vertx.max-event-loop-execute-time=10s

conductor.app.systemTaskWorkerThreadCount=200
conductor.app.systemTaskMaxPollCount=200
conductor.system-task-workers.poll-interval=10ms
conductor.app.sweeperThreadCount=32
conductor.app.sweeper.sweepBatchSize=50
conductor.app.workflowOffsetTimeout=1s

conductor.elasticsearch.indexBatchSize=500
conductor.elasticsearch.asyncWorkerQueueSize=5000
conductor.elasticsearch.indexRefreshInterval=5s
```

6. Lokasi File & Backup di Workspace

* File Workflow Hasil Optimasi: C:/Users/sony.wibisono/source/conductor/sony/workflows.json (Sudah di-import ke Conductor)

* File Backup Workflow Asli: C:/Users/sony.wibisono/source/conductor/sony/workflows.json.bak